

IT3071 – Machine Learning and Optimization Methods

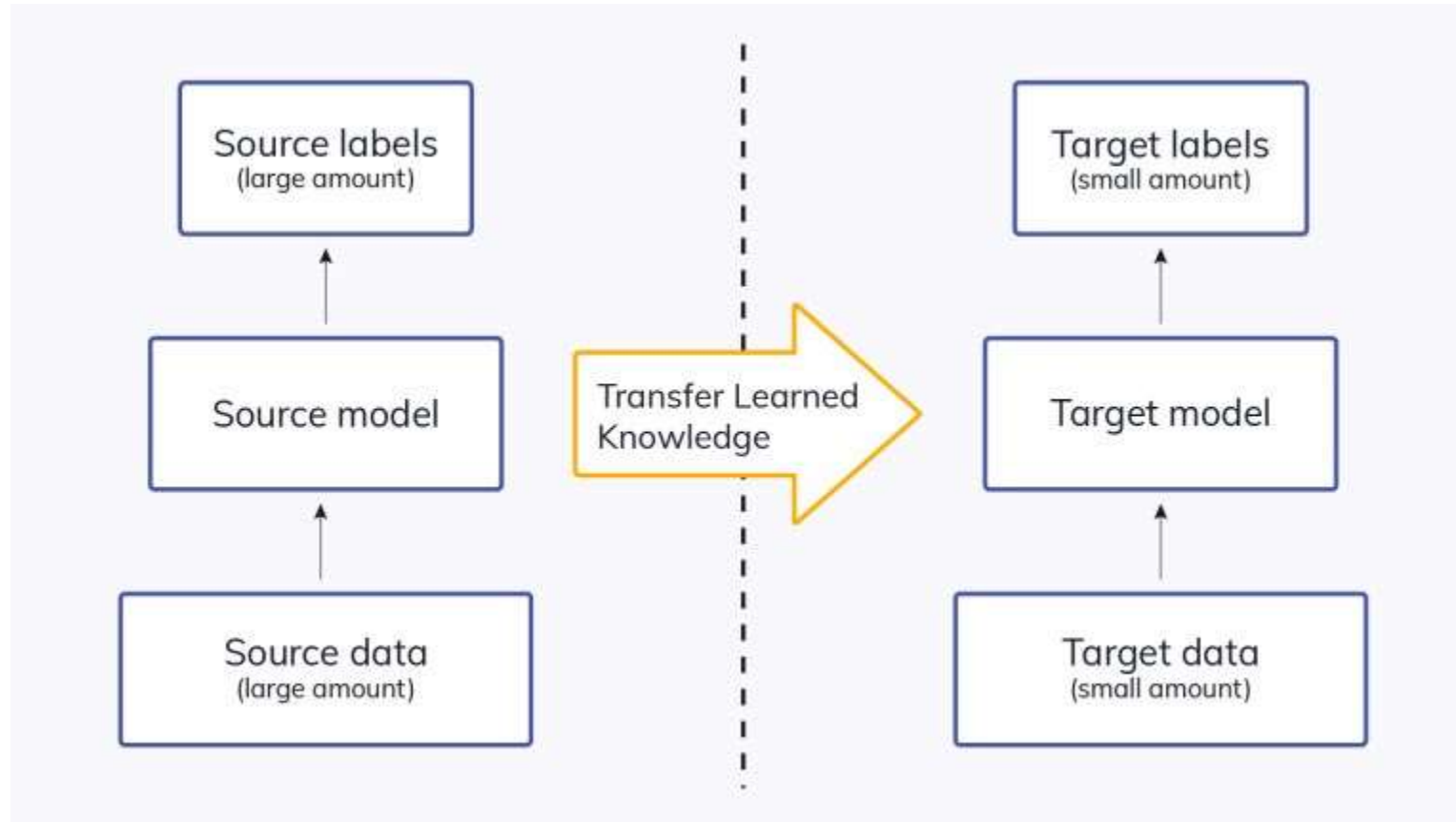
Introduction to Transfer Learning

H.M. Samadhi Chathuranga Rathnayake

What is Transfer Learning

- A model trained on one task is repurposed on a second, related task as an optimization that allows rapid progress when modeling the second task.
- By applying transfer learning to a new task, one can achieve significantly higher performance than training with only a small amount of data.
- Transfer learning is about leveraging feature representations from a pre-trained model, so you don't have to train a new model from scratch.
- The pre-trained models are usually trained on massive datasets that are a standard benchmark in the computer vision frontier.
- The weights obtained from the models can be reused in other tasks.

What is Transfer Learning



Traditional Machine Learning VS Transfer Learning

- Traditional machine learning models require training from scratch, which is computationally expensive and requires a large amount of data to achieve high performance.
- On the other hand, transfer learning is computationally efficient and helps achieve better results using a small data set.

Traditional Machine Learning VS Transfer Learning

- Traditional ML has an isolated training approach where each model is independently trained for a specific purpose, without any dependency on past knowledge.
- Contrary to that, transfer learning uses knowledge acquired from the pre-trained model to proceed with the task.

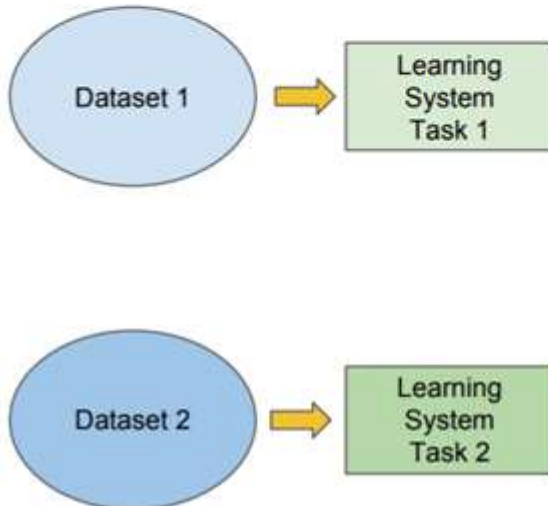
Traditional Machine Learning VS Transfer Learning

- Transfer learning models achieve optimal performance faster than the traditional ML models.
- It is because the models that leverage knowledge (features, weights, etc.) from previously trained models already understand the features.
- It makes it faster than training neural networks from scratch.

Traditional Machine Learning VS Transfer Learning

Traditional ML

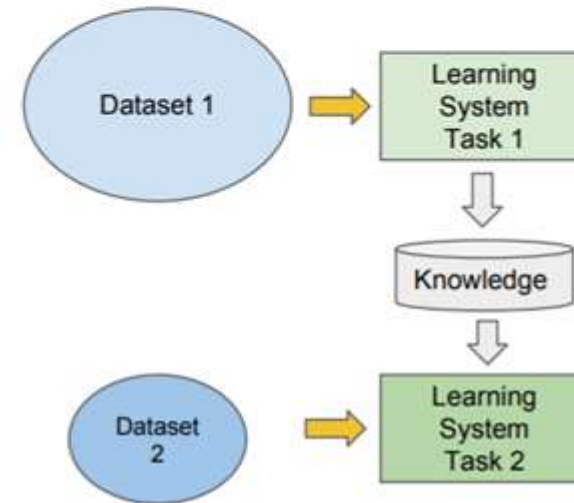
- Isolated, single task learning:
 - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

Transfer Learning

- Learning of a new task relies on the previous learned tasks:
 - Learning process can be faster, more accurate and/or need less training data



Classical Transfer Learning Strategies

- Different transfer learning strategies and techniques are applied based on the domain of the application, the task at hand, and the availability of data.
- Before deciding on the strategy of transfer learning, it is crucial to have an answer of the following questions:
 - **Which** part of the knowledge can be transferred from the source to the target to improve the performance of the target task?
 - **When** to transfer and when not to, so that one improves the target task performance/results and does not degrade them?
 - **How** to transfer the knowledge gained from the source model based on our current domain/task?
- Traditionally, transfer learning strategies fall under three major categories depending upon the task domain and the amount of labeled/unlabeled data present.
 - Inductive Transfer Learning
 - Transductive Transfer Learning
 - Unsupervised Transfer Learning

Inductive Transfer Learning

- Inductive Transfer Learning requires the source and target domains to be the same, though the specific tasks the model is working on are different.
- The algorithms try to use the knowledge from the source model and apply it to improve the target task.
- The pre-trained model already has expertise on the features of the domain and is at a better starting point than if we were to train it from scratch.
- Inductive transfer learning is further divided into two subcategories depending upon whether the source domain contains labeled data or not.
- These include multi-task learning and self-taught learning, respectively.
- Ex-An example of inductive transfer learning is training a model for object detection in one domain (e.g., cars in traffic images) and then transferring the learned knowledge to detect objects in a different domain (e.g., pedestrians in urban settings). Although the tasks differ, the general ability to detect objects can be transferred.

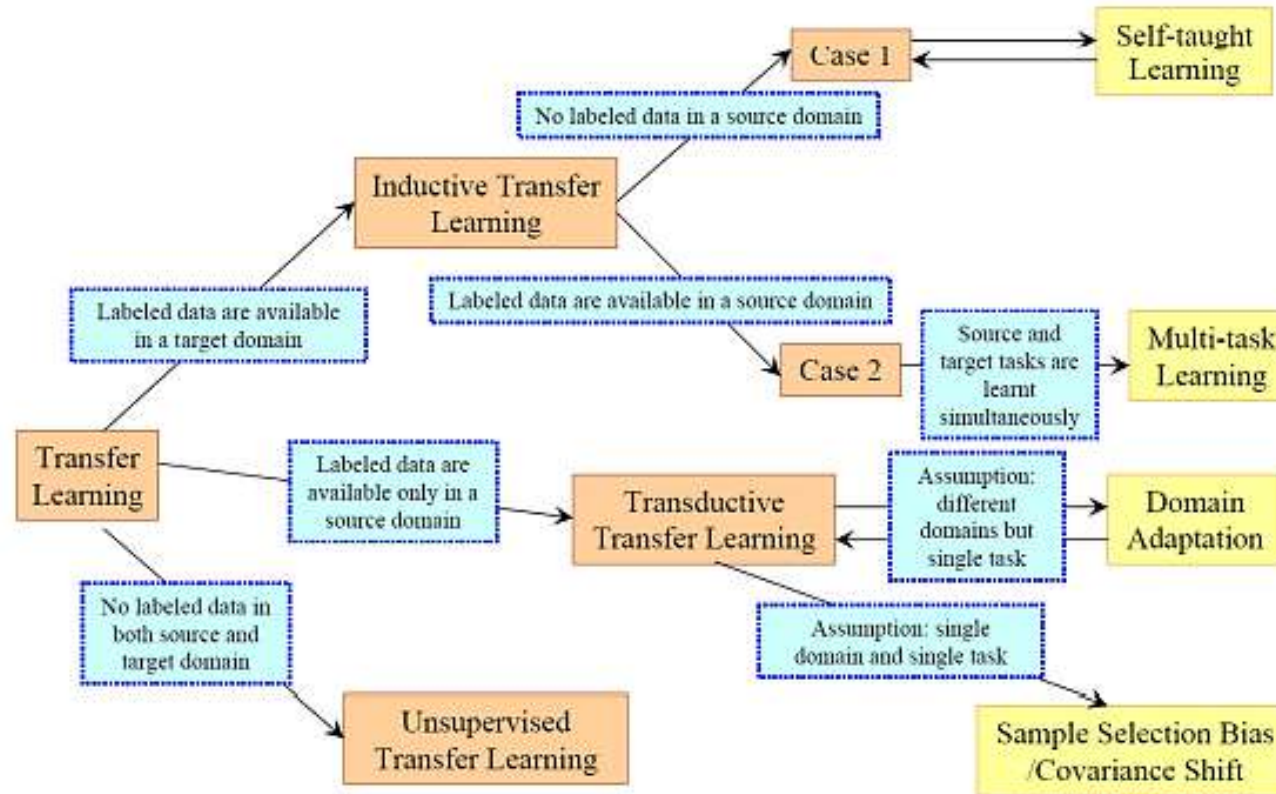
Transductive Transfer Learning

- Scenarios where the domains of the source and target tasks are not exactly the same but interrelated uses the Transductive Transfer Learning strategy.
- One can derive similarities between the source and target tasks.
- These scenarios usually have a lot of labeled data in the source domain, while the target domain has only unlabeled data.
- Ex-An example of transductive transfer learning could be using a sentiment analysis model trained on English movie reviews to classify sentiments in French movie reviews. Although the task (sentiment analysis) is the same, the domains (languages) differ, and the model needs to adapt its learned knowledge to perform well on the target domain (French).

Unsupervised Transfer Learning

- Unsupervised Transfer Learning is similar to Inductive Transfer learning.
- The only difference is that the algorithms focus on unsupervised tasks and involve unlabeled datasets both in the source and target tasks.
- Ex- An example of unsupervised transfer learning is using a model trained on labeled images of objects (source domain) to perform clustering on a different set of unlabeled images (target domain). Here, the learned representations or features from the source task help the model group similar objects together in the target domain, even though the target data has no labels.

Classical Transfer Learning Strategies



Approaches of Transfer Learning

- Now, we'll go through another way of categorizing transfer learning strategies based on the similarity of the domain and independent of the type of data samples present for training.
 - Homogeneous Transfer Learning
 - Heterogeneous Transfer Learning

Homogeneous Transfer Learning

- Homogeneous Transfer learning approaches are developed and proposed to handle situations where the domains are of the same feature space.
- In Homogeneous Transfer learning, domains have only a slight difference in marginal distributions.
- These approaches adapt the domains by correcting the sample selection bias or covariate shift.

Instance Transfer

- It covers a simple scenario in which there is a large amount of labeled data in the source domain and a limited number in the target domain.
- Both the domains and feature spaces differ only in marginal distributions.
- For example, suppose we need to build a model to diagnose cancer in a specific region where the elderly are the majority.
- Limited target-domain instances are given, and relevant data are available from another region where young people are the majority.
- Directly transferring all the data from another region may be unsuccessful since the marginal distribution difference exists, and the elderly have a higher risk of cancer than younger people.
- In this scenario, it is natural to consider adapting the marginal distributions.
- Instance-based Transfer learning reassigns weights to the source domain instances in the loss function.

Parameter Transfer

- The parameter-based transfer learning approaches transfer the knowledge at the model/parameter level.
- This approach involves transferring knowledge through the shared parameters of the source and target domain learner models.
- One way to transfer the learned knowledge can be by creating multiple source learner models and optimally combining the re-weighted learners similar to ensemble learners to form an improved target learner.
- The idea behind parameter-based methods is that a well-trained model on the source domain has learned a well-defined structure, and if two tasks are related, this structure can be transferred to the target model.

Parameter Transfer

- In general, there are two ways to share the weights in deep learning models: soft weight sharing and hard weight sharing.
 - In **soft weight sharing**, the model is expected to be close to the already learned features and is usually penalized if its weights deviate significantly from a given set of weights.
 - In **hard weight sharing**, we share the exact weights among different models.

Feature-Representation Transfer

- Feature-based approaches transform the original features to create a new feature representation.
- This approach can further be divided into two subcategories, i.e., asymmetric and symmetric Feature-based Transfer Learning.
 - **Asymmetric approaches** transform the source features to match the target ones. In other words, we take the features from the source domain and fit them into the target feature space. There can be some information loss in this process due to the marginal difference in the feature distribution.
 - **Symmetric approaches** find a common latent feature space and then transform both the source and the target features into this new feature representation.

Relational-Knowledge Transfer

- Relational-based transfer learning approaches mainly focus on learning the relations between the source and a target domain and using this knowledge to derive past knowledge and use it in the current context.
- Such approaches transfer the logical relationship or rules learned in the source domain to the target domain.
- For example, if we learn the relationship between different elements of the speech in a male voice, it can help significantly to analyze the sentence in another voice.

Heterogeneous Transfer Learning

- Transfer learning involves deriving representations from a previous network to extract meaningful features from new samples for an inter-related task.
- However, these approaches forget to account for the difference in the feature spaces between the source and target domains.
- It is often challenging to collect labeled source domain data with the same feature space as the target domain, and Heterogeneous Transfer learning methods are developed to address such limitations.
- This technique aims to solve the issue of source and target domains having differing feature spaces and other concerns like differing data distributions and label spaces.
- Heterogeneous Transfer Learning is applied in cross-domain tasks such as cross-language text categorization, text-to-image classification, and many others.

Transfer Learning for Deep Learning

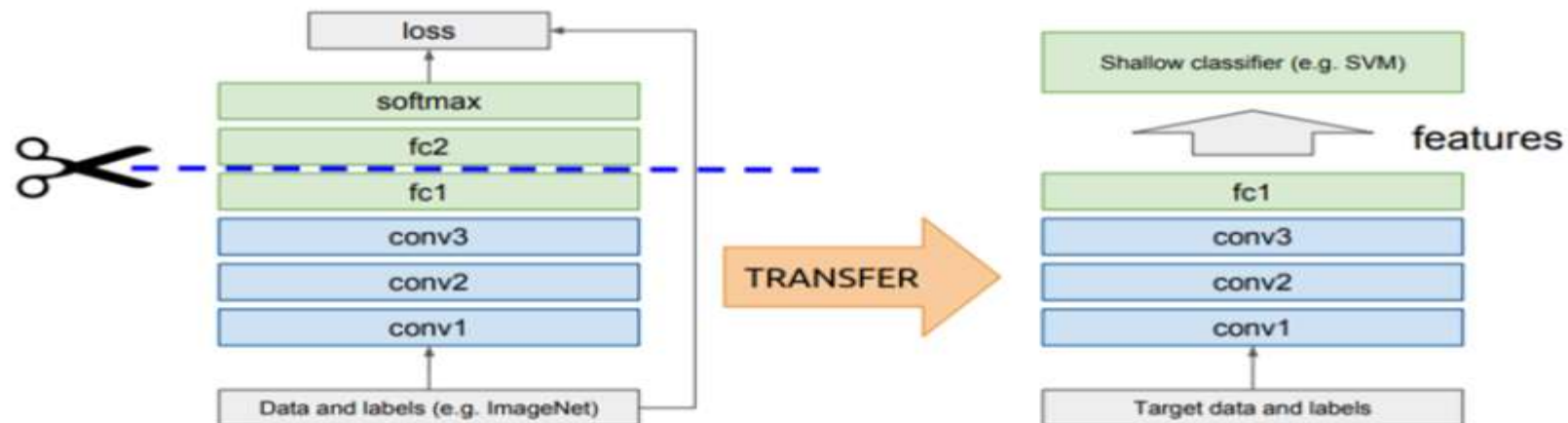
Off-the-shelf Pre-trained Models as Feature Extractors

- Deep learning systems are layered architectures that learn different features at different layers.
- Initial layers compile higher-level features that narrow down to fine-grained features as we go deeper into the network.
- These layers are finally connected to the last layer (usually a fully connected layer, in the case of supervised learning) to get the final output.
- This opens the scope of using popular pre-trained networks (such as Oxford VGG Model, Google Inception Model, Microsoft ResNet Model) without its final layer as a fixed feature extractor for other tasks.

Off-the-shelf Pre-trained Models as Feature Extractors

- The key idea here is to leverage the pre-trained model's weighted layers to extract features, but not update the model's weights during training with new data for the new task.
- The pre-trained models are trained on a large and general enough dataset and will effectively serve as a generic model of the visual world.

Assumes that $D_S = D_T$



Fine Tuning Off-the-shelf Pre-trained Models

- This is a more engaging technique, where we do not just directly depend on the features extracted from the pre-trained models and replace the final layer but also selectively retrain some of the previous layers.
- Deep neural networks are layered structures and have many tunable hyperparameters.
- The role of the initial layers is to capture generic features, while the later ones focus more on the explicit task at hand.
- It makes sense to fine-tune the higher-order feature representations in the base model to make them more relevant for the specific task.
- We can re-train some layers of the model while keeping some frozen in training.

Fine Tuning Off-the-shelf Pre-trained Models

- An example is depicted in the following figure on an object detection task, where initial lower layers of the network learn very generic features and the higher layers learn very task-specific features.

Fine-tuning: supervised domain adaptation

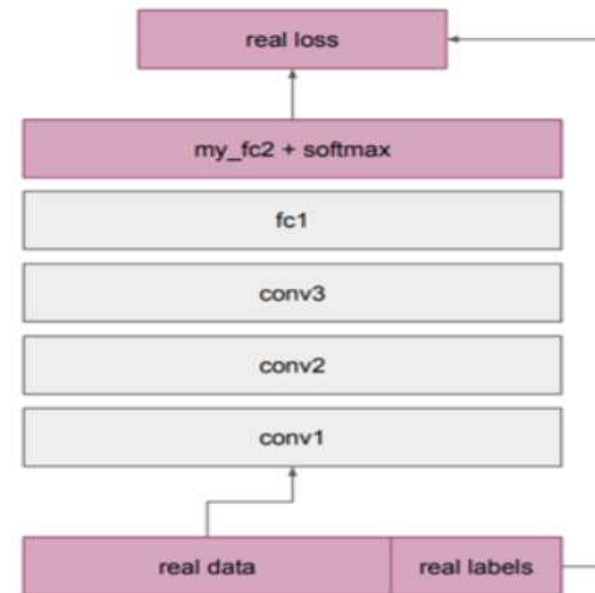
Train deep net on “nearby” task for which it is easy to get labels using standard backprop

- E.g. ImageNet classification
- Pseudo classes from augmented data
- Slow feature learning, ego-motion

Cut off top layer(s) of network and replace with supervised objective for target domain

Fine-tune network using backprop with labels for target domain until validation loss starts to increase

Aligns D_S with D_T



Freezing vs. Fine-tuning

- One logical way to increase the model's performance even further is to re-train (or "fine-tune") the weights of the top layers of the pre-trained model alongside the training of the classifier you added.
- This will force the weights to be updated from generic feature maps the model has learned from the source task.
- Fine-tuning will allow the model to apply past knowledge in the target domain and re-learn some things again.
- Moreover, one should try to fine-tune a small number of top layers rather than the entire model.
- The first few layers learn elementary and generic features that generalize to almost all types of data.

Freezing vs. Fine-tuning

- Therefore, it's wise to freeze these layers and reuse the basic knowledge derived from the past training.
- As we go higher up, the features are increasingly more specific to the dataset on which the model was trained.
- Fine-tuning aims to adapt these specialized features to work with the new dataset, rather than overwrite the generic learning.

Freezing vs. Fine-tuning

Freeze or fine-tune?

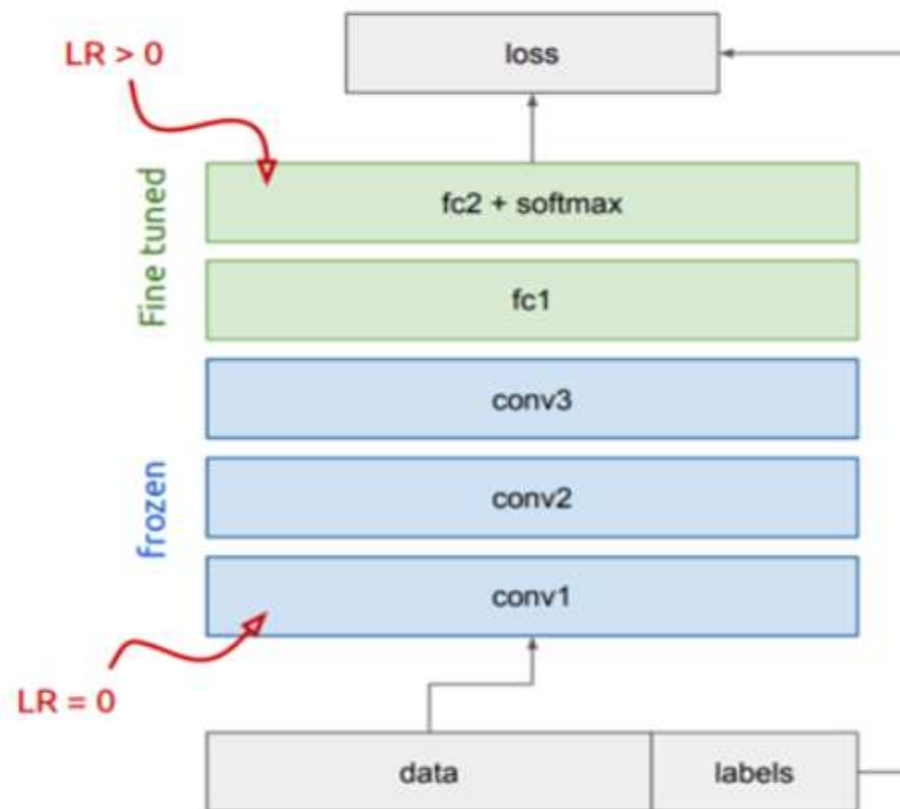
Bottom n layers can be frozen or fine tuned.

- **Frozen:** not updated during backprop
- **Fine-tuned:** updated during backprop

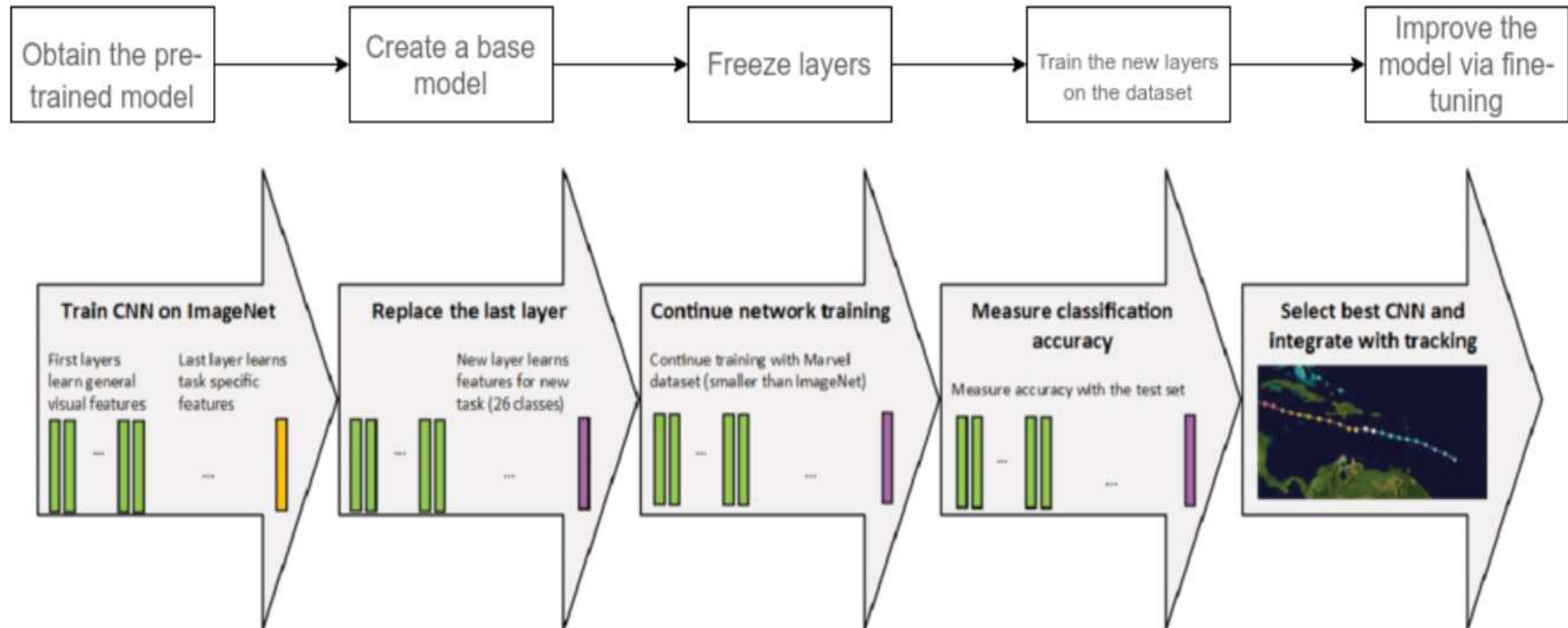
Which to do depends on target task:

- **Freeze:** target task labels are scarce, and we want to avoid overfitting
- **Fine-tune:** target task labels are more plentiful

In general, we can set learning rates to be different for each layer to find a tradeoff between freezing and fine tuning



Steps of Transfer Learning



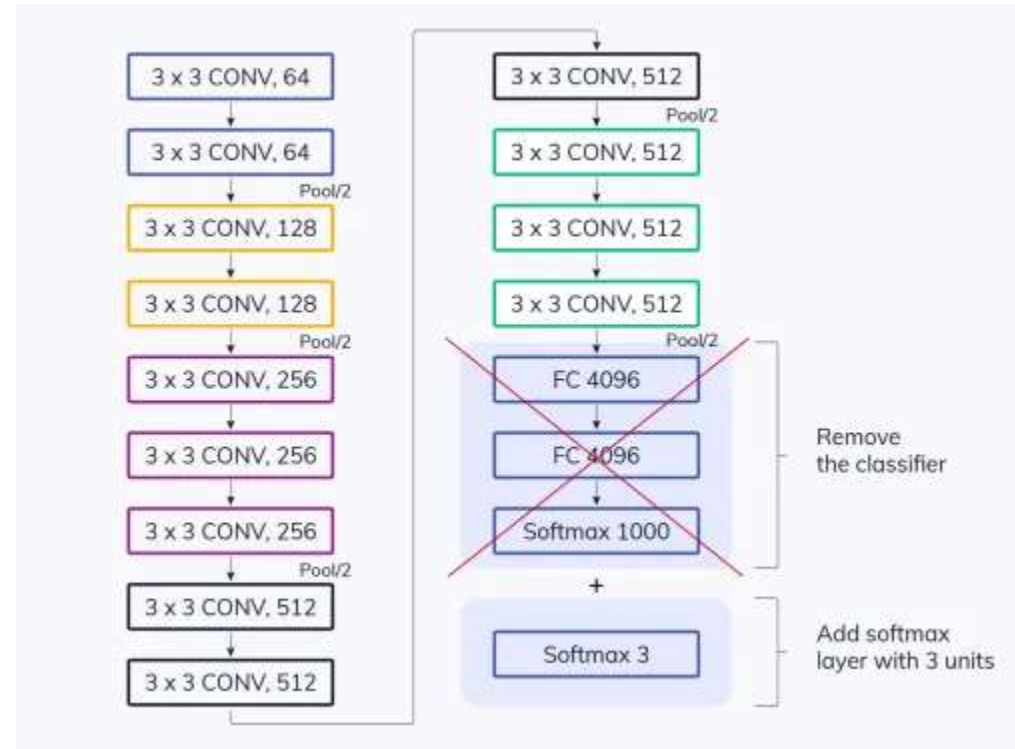
Obtain pre-trained model

- The first step is to choose the pre-trained model we would like to keep as the base of our training, depending on the task.
- Transfer learning requires a strong correlation between the knowledge of the pre-trained source model and the target task domain for them to be compatible.
- For computer vision:
 - VGG-16
 - VGG-19
 - Inception V3
 - Xception
 - ResNet-50
- For NLP tasks:
 - Word2Vec
 - GloVe
 - FastText

Create a base model

- The base model is one of the architectures such as ResNet or Xception which we have selected in the first step to be in close relation to our task.
- We can either download the network weights which saves the time of additional training of the model.
- Else, we will have to use the network architecture to train our model from scratch.
- There can be a case where the base model will have more neurons in the final output layer than we require in our use case.
- In such scenarios, we need to remove the final output layer and change it accordingly.

Create a base model



Freeze layers

- Freezing the starting layers from the pre-trained model is essential to avoid the additional work of making the model learn the basic features.
- If we do not freeze the initial layers, we will lose all the learning that has already taken place.
- This will be no different from training the model from scratch and will be a loss of time, resources, etc.

Add new trainable layers

- The only knowledge we are reusing from the base model is the feature extraction layers.
- We need to add additional layers on top of them to predict the specialized tasks of the model.
- These are generally the final output layers.

Train the new layers

- The pre-trained model's final output will most likely differ from the output we want for our model.
- For example, pre-trained models trained on the ImageNet dataset will output 1000 classes.
- However, we need our model to work for two classes.
- In this case, we have to train the model with a new output layer in place.

Fine-tune your model

- One method of improving the performance is fine-tuning.
- Fine-tuning involves unfreezing some part of the base model and training the entire model again on the whole dataset at a very low learning rate.
- The low learning rate will increase the performance of the model on the new dataset while preventing overfitting.